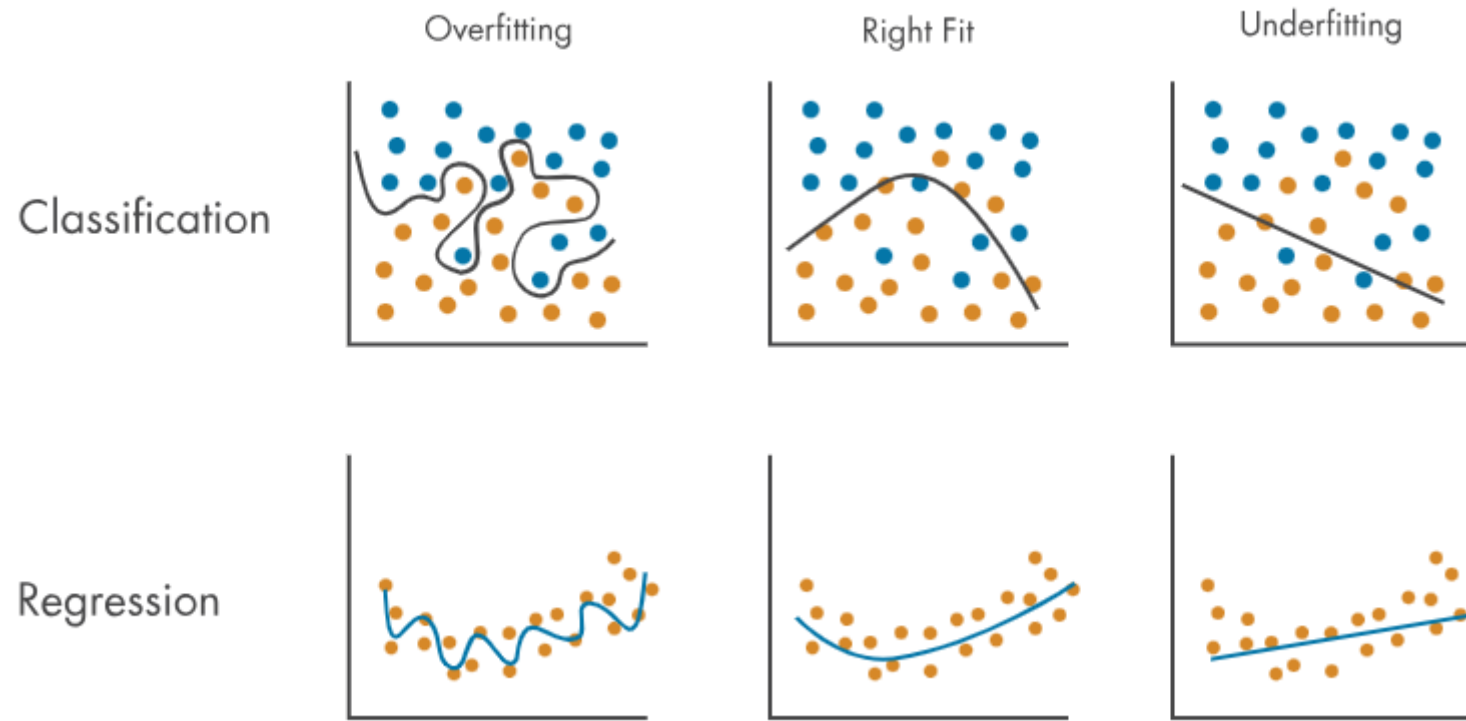


RETELE NEURONALE

PARTEA a II-a

Overfitting (supra-ajustare)

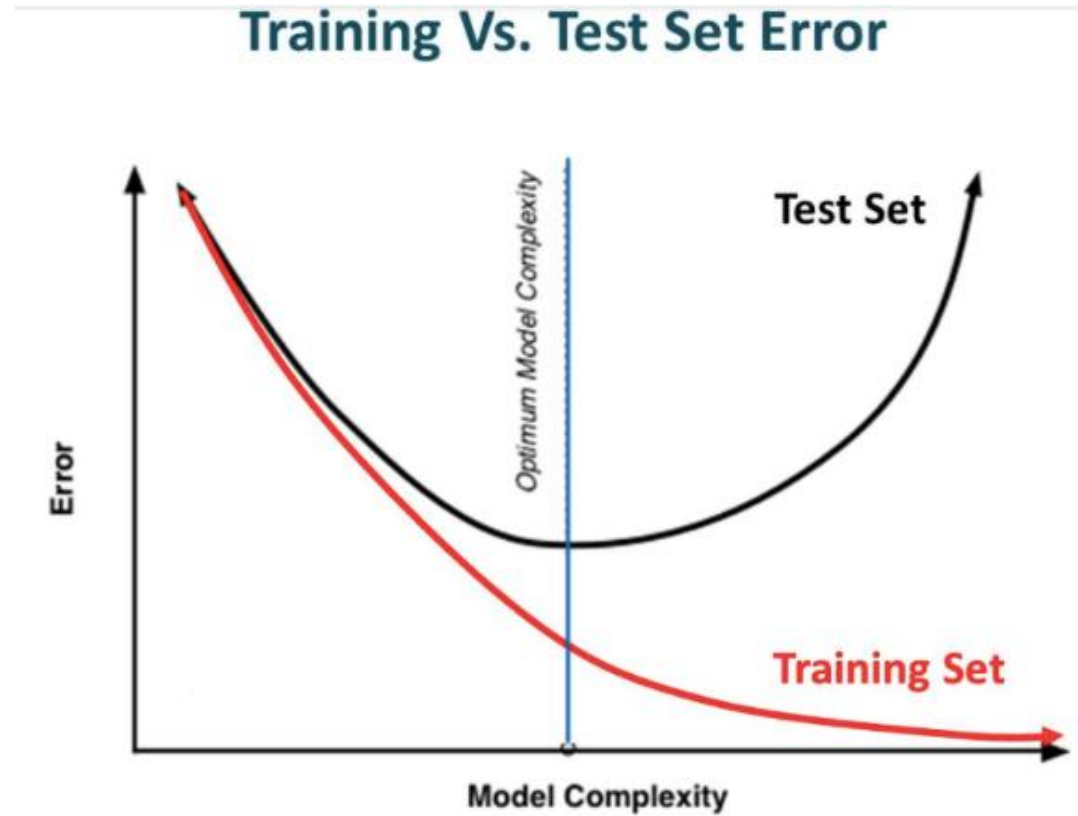
- Overfitting apare atunci când modelul are o acuratețe foarte bună pe set de training adică a învățat atât de bine set de training încât nu știe cum să răspundă la datele noi, cum sunt cele din mulțimea de testare.
- Poate să apară dacă:
 - Modelul de ML este prea complex;
 - Memorează trăsături subtile în mulțimea de training care nu se pot generaliza.
 - Dimensiunea mulțimii de training este prea mică pentru complexitatea modelului și/sau conține cantități mari de informații irelevante.



Regularizare

- O tehnică pe care o putem introduce in retea neuronală pentru a descuraja învățarea modelelor prea complexe
- Modificarea algoritmului de invatare a.i. modelul generalizeaza mai bine si astfel este imbunatatita performanta.

Early stopping



Source: Slideplayer

```
from keras.callbacks import EarlyStopping EarlyStopping(monitor='val_err', patience=5)
```

<https://www.analyticsvidhya.com/blog/2018/04/fundamentals-deep-learning-regularization-techniques/>

Multimea de validare

- Multimea de training se imparte in multime de antrenare si multime de validare (De ex. Folosim 20% din multimea de antrenare ca multime de validare.)
- Multimea de validare acționează ca un punct de control, permițând detectarea overfitting-ului (supra-ajustarii).
- Multimea de validare este un set de date care este utilizat pentru a valida performanța modelului în timpul antrenamentului.
- La fiecare iteratie (epoca) se evalueaza performanța modelului pe multimea de validare, putand ajusta complexitatea modelului, tehnicile sau alți hiperparametri pentru a preveni overfitting -ul și a îmbunătăți generalizarea.
- După fiecare epocă, modelul este antrenat pe setul de antrenament, iar evaluarea modelului este efectuată pe setul de validare.

- `model.fit(x_train, y_train, epochs=10)`
- Vs.
- `model.fit(x_train, y_train, validation_split = 0.2, epochs=10)`

Dropout

- cea mai frecvent utilizată tehnică de regularizare în domeniul DL
- Foarte interesanta si produce rezultate foarte bune
- La fiecare iterație, selectează aleatoriu unele noduri și le elimină împreună cu toate conexiunile lor de intrare și de ieșire.

Dropout

- In codul de mai jos 20% din noduri vor fi eliminate.
-
- `model = tf.keras.models.Sequential([`
- `tf.keras.layers.Flatten(input_shape=(28, 28)),`
- `tf.keras.layers.Dense(128, activation='relu'),`
- **`tf.keras.layers.Dropout(0.2),`**
- `tf.keras.layers.Dense(10)`
- `])`